Verslag labo 2

Opdracht

Voor deze opdracht werd er gevraagd om op drie verschillende manieren een maximale waarde uit een willekeurige 1D array te lezen. De eerste manier was door middel van een nested for-loop via de CPU, de tweede manier door middel van atomic operations via de GPU en tenslotte door reduction via de GPU. Als bijkomende parameter werd ook telkens gevraagd om weer te geven hoelang het duurde om deze bewerkingen uit te voeren.

Werkwijze

De werkwijze die werd gehanteerd was door stap per stap te werken en de code te schrijven. Eens de code werkt, werd deze uitgebreid door de timing-parameters eraan te toe voegen. Pas als het volledige deel afgewerkt is werd naar de volgende vraag gegaan.

Code

Deel 1: Hier werd een random array van een bepaalde grootte N aangemaakt die random waarden bevat tussen de 0 en 99. Deze werd door middel van een for-loop uitgelezen waarbij een externe parameter maxValue kijkt of de ogenblikkelijke waarde groter is dan de waarde in maxValue. Als dit zo is wordt deze waarde in maxValue gestoken. Zo verkrijgen we uiteindelijk de maximale waarde uit de array.

Deel 2: Ten eerste maken we terug een random array aan, hierna alloceren we plaats op de GPU en kopiëren we de array naar deze plaats. Belangrijk hierbij is dat bij de cudaMemcpyHostToDevice we eerst de pointer meegeven en vervolgens de geheugenplaats van de array waar bij cudaMemcpyDeviceToHost eerst de geheugenplaats van de array weergeeft en als tweede parameter de pointer. In de __global__ void findMax (met parameters 2 pointers van integers en een integer als 3de parameter) bepalen we eerst de index van het aantal bewerkingen dat er moeten worden uitgevoerd. Vervolgens wordt de atomicMax-functie uitgevoerd waarbij per index wordt gekeken of deze groter is en zo ja, wordt deze onthouden. De eindwaarde wordt in de pointer result gestoken om deze daarna terug naar de CPU te kunnen schrijven. Merk op dat we zowel een aparte pointer *result kunnen gebruiken alsook de eerste waarde van de array (namelijk array[0]). Beide methoden zijn correct om het resultaat uit te lezen.

Deel 3: Voor dit deel wordt de oproep van de __global__ void findMax in een while-loop gestoken waarbij deze telkens kijkt naar de grootte van de array. Na elke uitvoering wordt deze gereduceerd met de helft waarbij op het einde enkel de maximale waarde overblijft op plaats array[0]. In findMax wordt telkens de value op plaats s vergeleken met plaats s+1 en de hoogste waarde onthouden

op plaats s en dit voor de gehele array. Vanaf de tweede keer worden stappen van 2s+1 genomen (dus 1,3,5,7,..).

Guiding questions

Is the result correct?

Het resultaat van de verkregen maximale waarde is correct waarbij we uit eender welke array telkens de maximale waarde verkrijgen op de output, zolang deze array maximum 1024 elementen bevat. Bij deel 3, waarbij we werken met reduction, is het afhankelijk welke array we gebruiken of dat deze werkt of niet.

How does time increase as a function of input size?

Theoretisch hadden we verwacht dat naarmate de lengte van de array stijgt, de tijd mee ging stijgen mits er meer bewerkingen worden uitgevoerd. Echter verkrijgen we bij de praktische waarden een minieme verandering van de tijd wanneer we de lengte van de array van 10 naar 10.000 veranderde.

What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?

Als we met een input moeten werken dat groter is dan 2048 elementen, zullen we met meerdere blocks moeten werken, aangezien er maar 1024 threads per block mogelijk zijn. Dit wil dus zeggen dat we moeten wachten tot alle blocks klaar zijn met hun 'eigen' maximum waarde te berekenen, voordat we nadien de maximum waarde nemen uit alle eindwaarden van iedere block.

Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

Bij het verminderen van het aantal threads ten opzichte van de hoeveelheid data-elementen, is een efficiënte aanpak cruciaal. Elke thread krijgt meerdere elementen toegewezen om te verwerken binnen zijn toegewezen deel. Het is belangrijk om grensgevallen te behandelen, zoals wanneer het aantal data-elementen niet gelijk verdeelbaar is door het aantal threads. Het synchroniseren van threads is van essentieel belang om racecondities en inconsistenties te voorkomen, vooral bij gedeeld gebruik van resources. Bovendien is het optimaliseren van geheugentoegang belangrijk. Ten slotte is het verstandig om de prestaties van de kernel te meten om ervoor te zorgen dat het verminderen van het aantal threads geen negatieve invloed heeft op de verwerkingssnelheid.

How would you tackle a 2D array?

Voor een 2D array kunnen we elke rij apart beschouwen. We berekenen voor elke rij de maximum waarde en steken deze op plaats 1 in de array van die rij. Hierna kunnen we de eerste kolom van de 2D array beschouwen als array en ook hiervan de maximum waarde berekenen zodanig we dus de maximum waarde uit de 2D array verkrijgen.